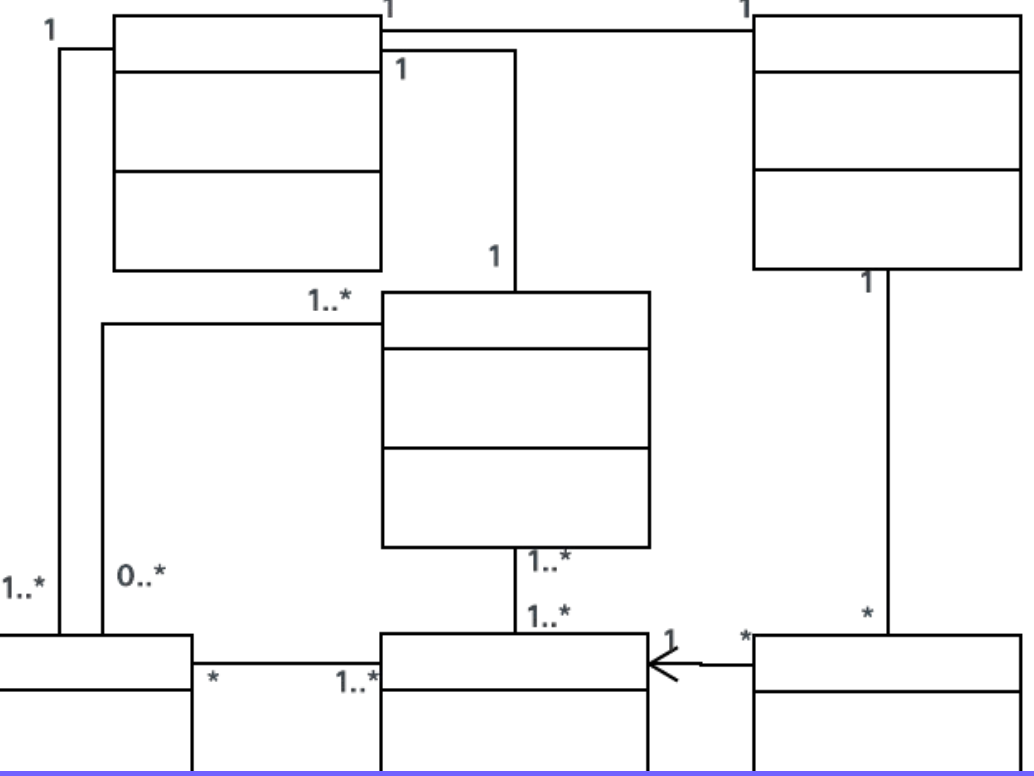
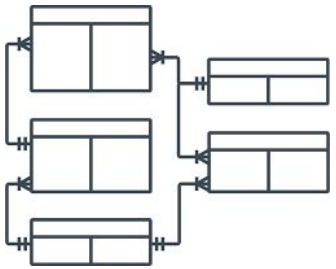








Diseño de Sistemas de Información

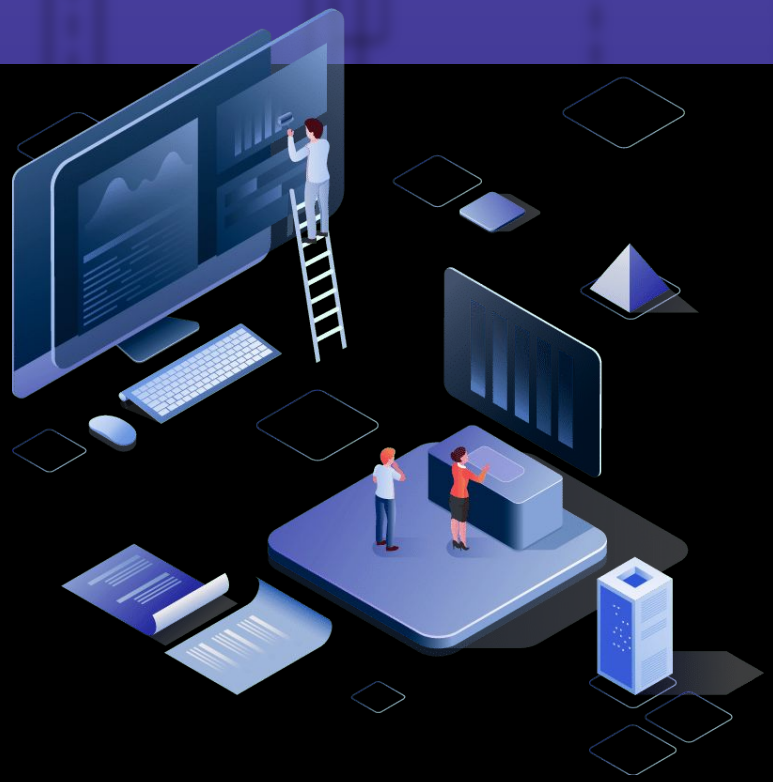




Agenda

- Introducción a la Arquitectura de Software.
- Niveles de la Arquitectura de Software.
- Requerimientos no funcionales como entrada de la Arquitectura. Otras entradas de la Arquitectura.
- Componentes típicos de los Sistemas de Información: Frontend, Backend, DB.
- Diagramas de Componentes y Despliegue.

Arquitectura de Software y de Sistema



Arquitectura de Software

Imaginá que vas a construir un edificio. Antes de poner el primer ladrillo, alguien tiene que decidir: ¿cuántos pisos va a tener? ¿Dónde van las escaleras? ¿Hay cochera? ¿Los caños de agua pasan por dónde?

Eso es exactamente lo que hace la arquitectura de software: son las decisiones grandes que tomás antes de escribir código, y que después son muy difíciles de cambiar.

Arquitectura de Software

La arquitectura de software representa la estructura o estructuras del sistema que consiste en componentes de software, las propiedades externas visibles de esos componentes y las relaciones entre ellos.

Arquitectura de Software

- Un componente es un **pieza de software** que puede ser código fuente, código binario, un ejecutable, o una biblioteca con una **interfaz definida**.

- Tiene su propio **ciclo de vida**: repositorio, equipo que lo mantiene, esquema de versionado, agenda de despliegues, etc…

- ¿Qué es la interfaz de un Componente?
  Una interfaz establece las operaciones externas de un componente, las cuales determinan una parte del comportamiento del mismo.

Arquitectura de Software

Ejemplos de interfaces de componentes

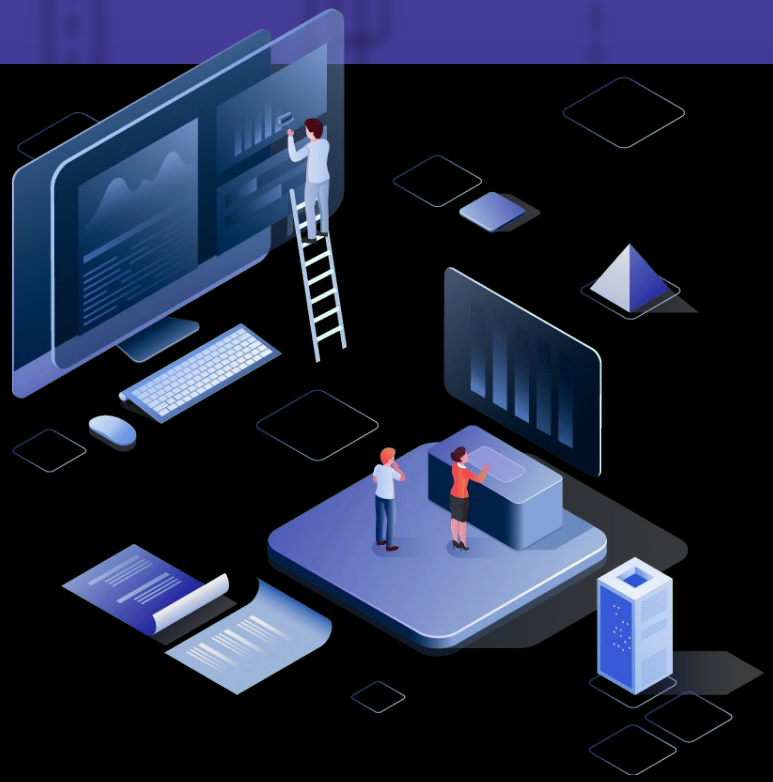
- El conjunto de mensajes que entiende un objeto
- El conjunto de métodos públicos utilizables de una biblioteca
- Una API REST expuesta por un Sistema
- Etc.

Arquitectura de Software

Analogía con ejemplo

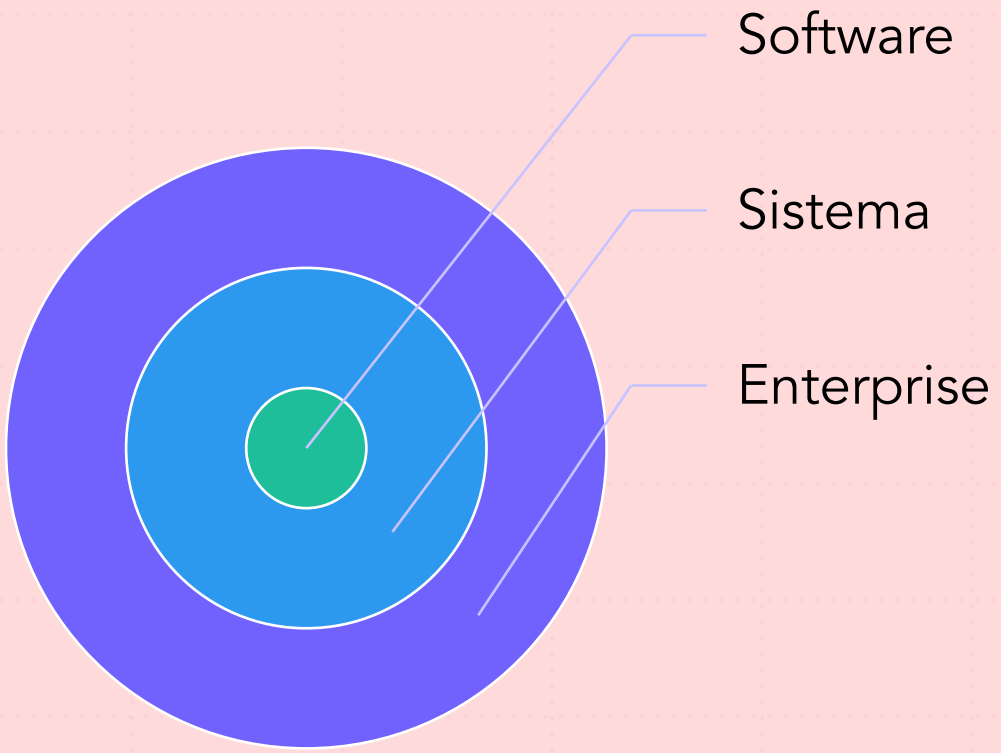
- *Edificio → Sistema de software*
- *Habitaciones / plantas → Componentes / módulos*
- *Puertas entre habitaciones → Interfaces entre componentes*
- *El plano del arquitecto → El diagrama de arquitectura*

Niveles y características de la Arquitectura



Niveles de la Arquitectura

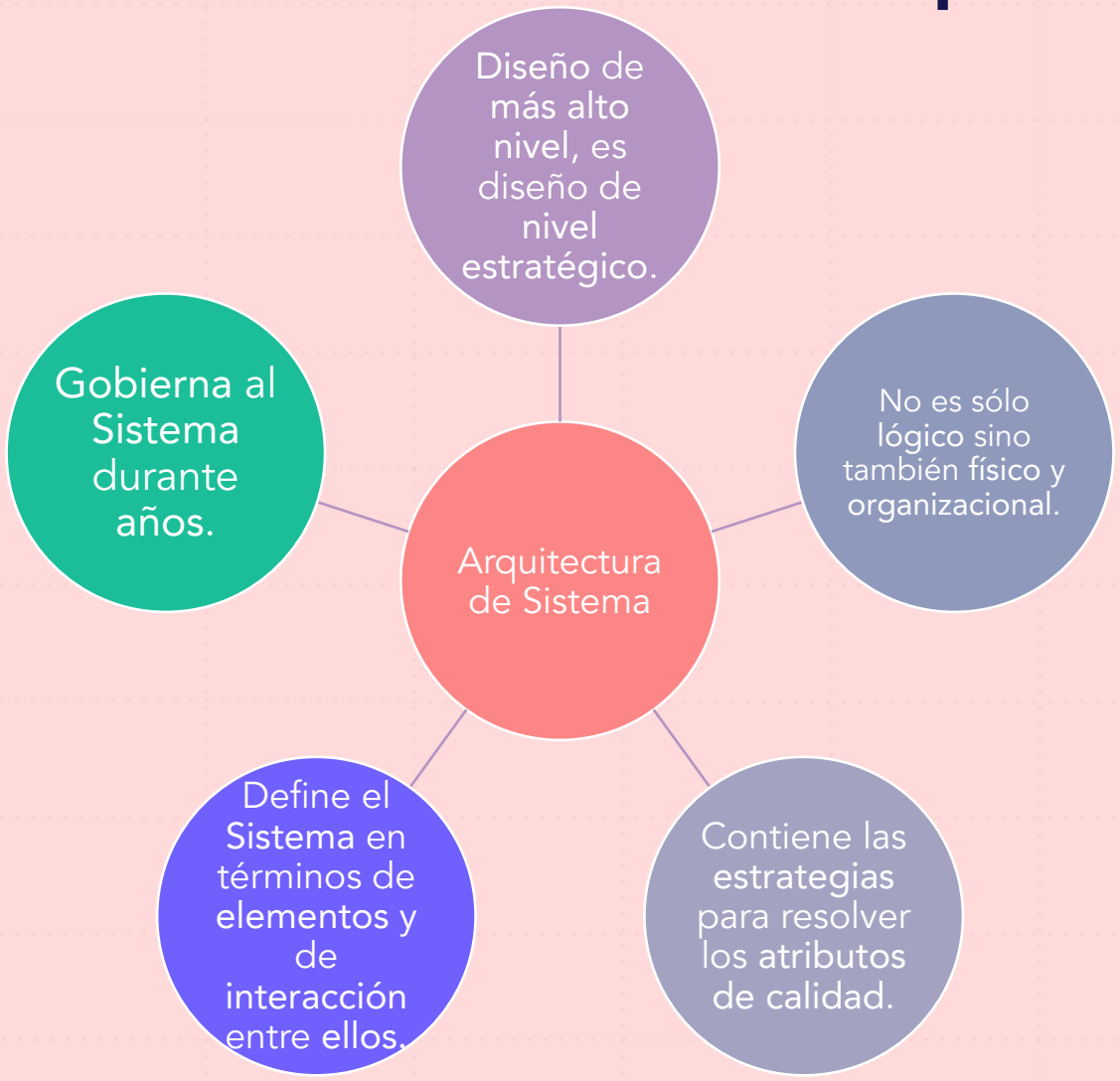
Enterprise	Sistema	Software
• Define la estrategia tecnológica y de negocio de la organización para el desarrollo de sus Sistemas.	• Arquitectura de Software e Infraestructura.	• Arquitectura de Software para una Aplicación o Subsistema.



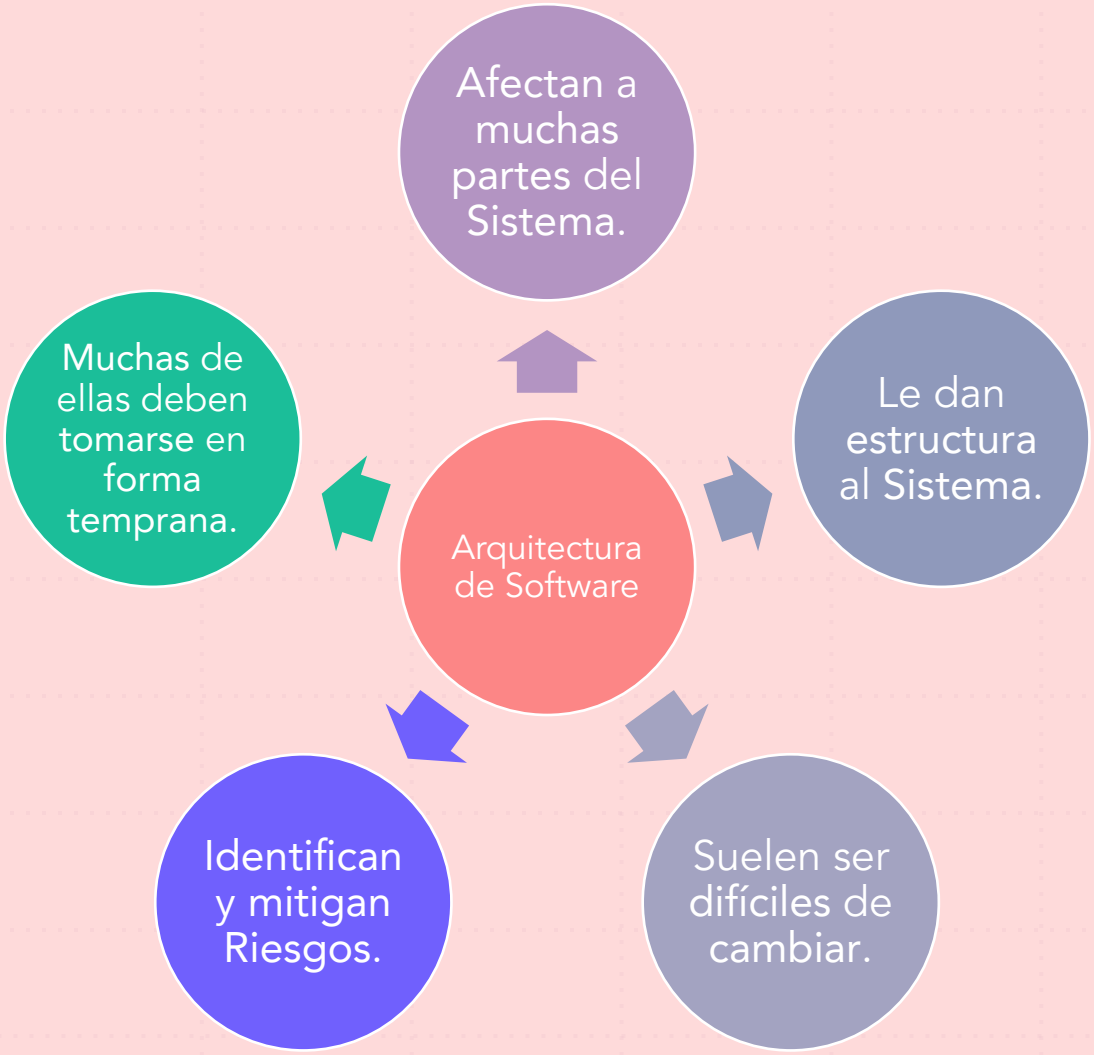
Niveles de la Arquitectura

Nivel	¿Qué se ve?	Ejemplo
Enterprise	Todos los sistemas de la organización y cómo se relacionan	"El sistema de Ventas habla con el de Stock y con el de Facturación"
Sistema	Un sistema y sus subsistemas principales	"Nuestra app tiene un Frontend, un Backend y una Base de Datos"
Software	Un componente específico por dentro	"El módulo de autenticación usa JWT y tiene estas clases..."

Características de la arquitectura



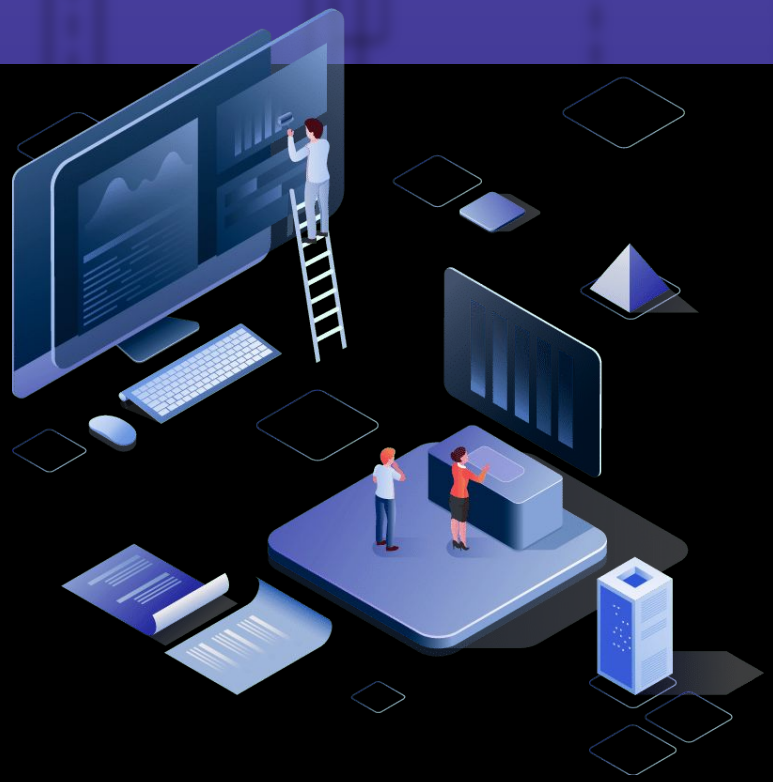
Decisiones arquitectónicas



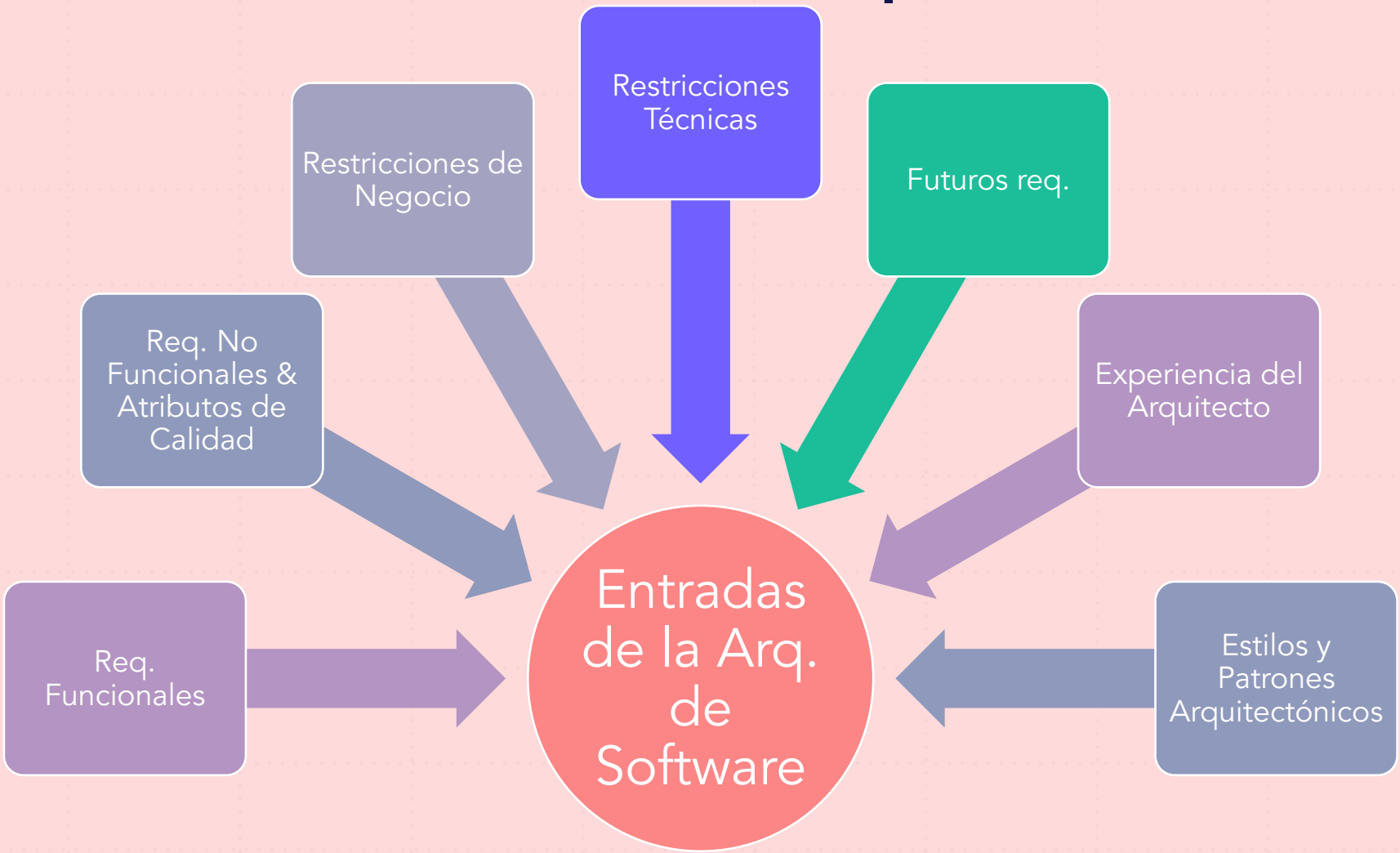
Decisiones arquitectónicas

Si el arquitecto se equivoca en el plano, derrumbar una pared después sale carísimo. En software, pasa exactamente lo mismo.

Entradas de la Arquitectura



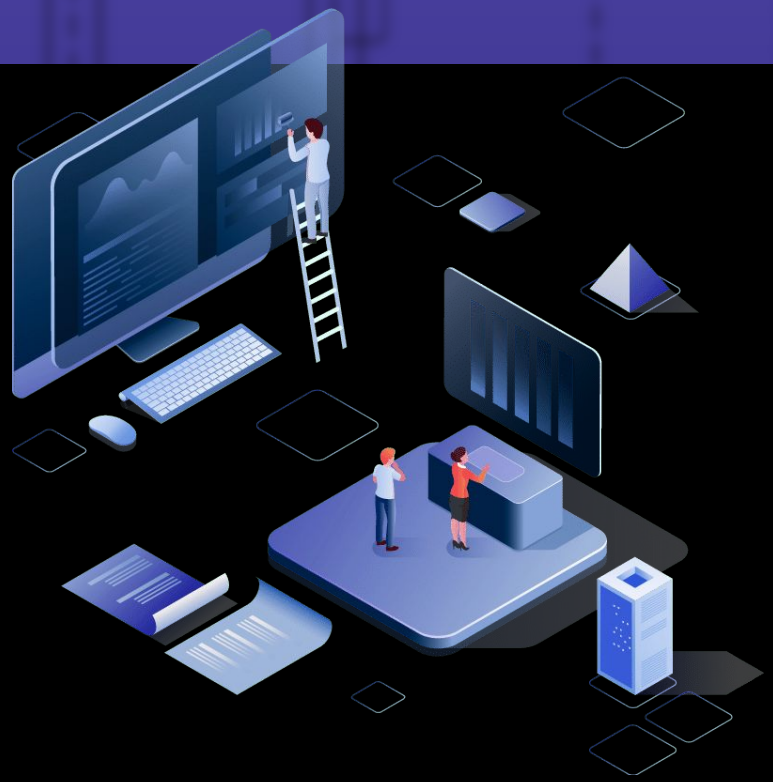
Entradas de la Arquitectura



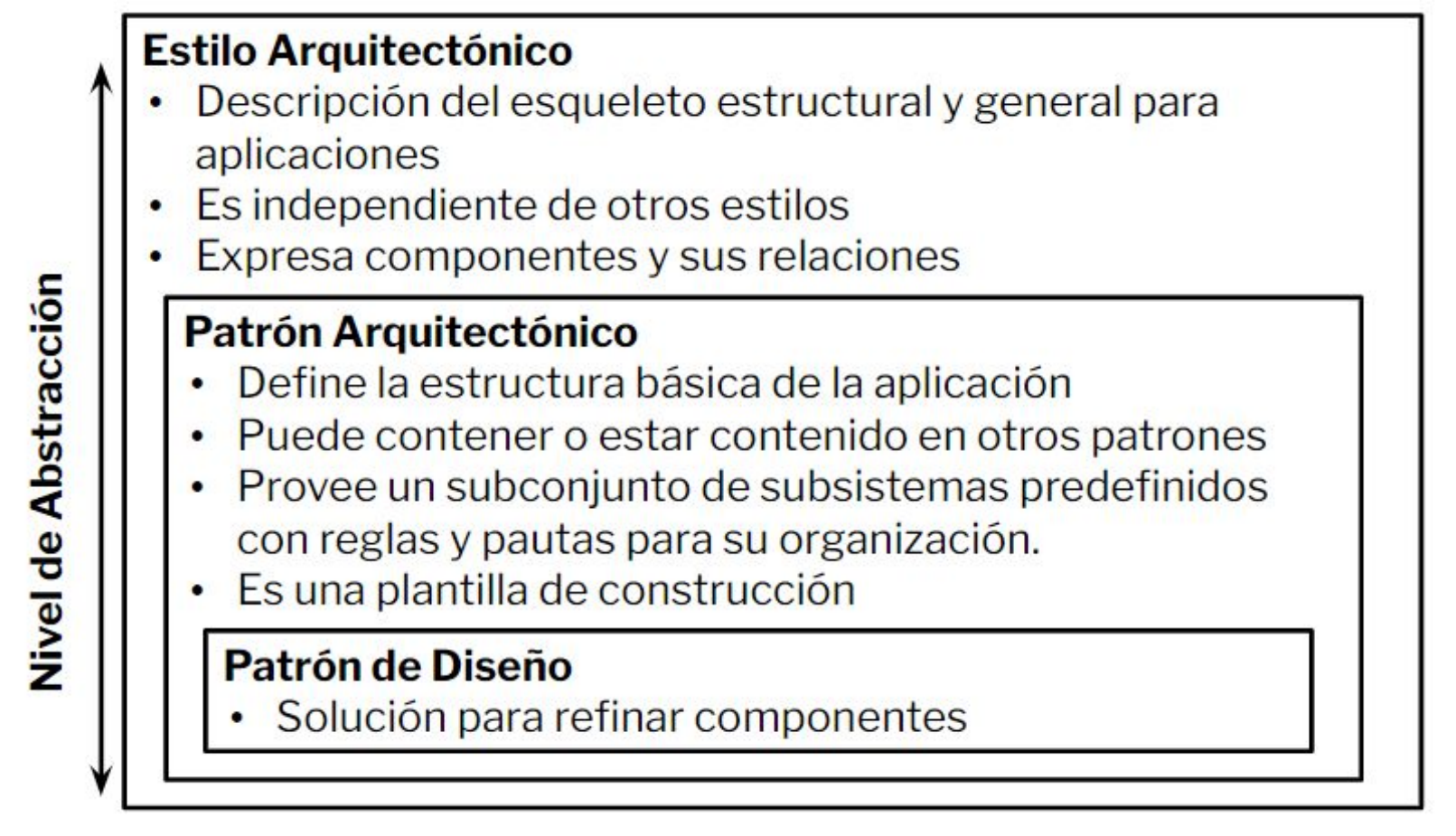
Entradas de la Arquitectura

Entrada	Ejemplo concreto
• Requerimientos Funcionales	• "El sistema debe permitir registrar pedidos de compra"
• Requerimientos No Funcionales / Atributos de Calidad	• "El sistema debe responder en menos de 2 segundos con 10.000 usuarios simultáneos"
• Restricciones Técnicas	• "Debe correr en la nube de Azure" / "El cliente ya tiene licencias de Oracle"
• Restricciones de Negocio	• "Debe estar listo en 6 meses" / "El presupuesto es de $50.000"
• Requerimientos Futuros	• "En 2 años necesitaremos abrir el sistema a clientes externos vía API"
• Experiencia del Arquitecto	• "Ya resolvimos algo similar con microservicios, sabemos que escala bien"
• Estilos y Patrones Arquitectónicos	• Microservicios, MVC, Event-Driven, Monolítico...

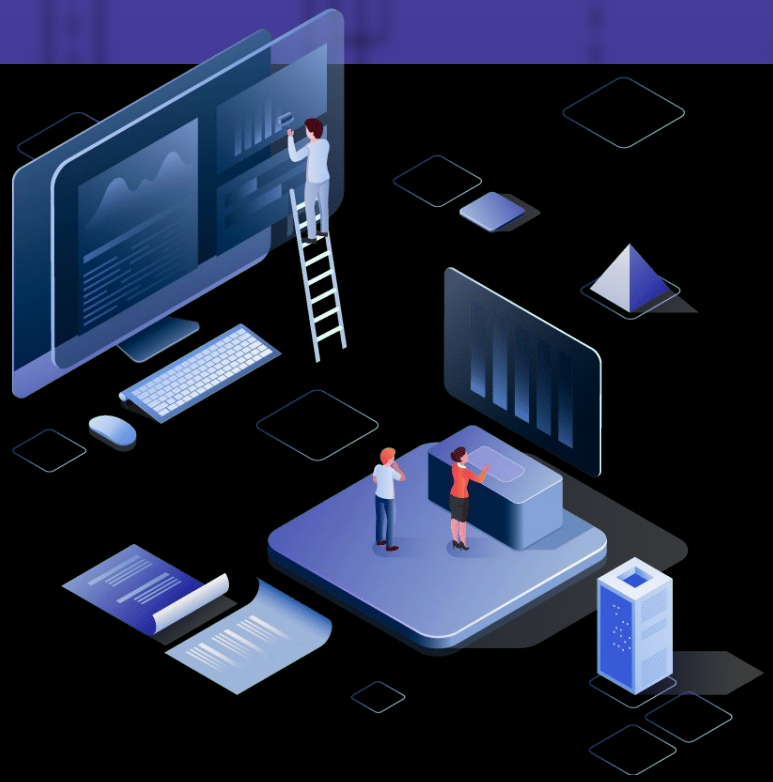
Niveles de abstracción



Arquitectura de Software – Niveles de Abstracción



Componentes típicos de un Sistema de Información



Componentes típicos

Muy a alto nivel podemos mencionar que en los Sistemas de Información principalmente nos encontraremos con dos grandes componentes arquitectónicos: Backend y Frontend.

Componentes típicos

Backend

- El backend es la capa de un Sistema de Software encargada del procesamiento de datos, gran parte de la lógica de negocio y la gestión de la comunicación entre el cliente (frontend) y la base de datos o servicios externos.
- El usuario nunca lo ve directamente.
- Funciona en el lado del servidor y generalmente expone una API que el frontend consume.
- Tecnologías típicas: Java, Python, PHP, etc.

Componentes típicos

Frontend

- El frontend es la capa de presentación gráfica del Sistema con la que interactúa el usuario.
- Es lo que el usuario ve y toca
- Se ejecuta en el navegador o en una aplicación cliente (aplicación mobile, por ejemplo) y se encarga de la presentación de la información y la experiencia de usuario.
- Tecnologías típicas: React, Angular, Vue / Swift, Kotlin

Componentes típicos

Base de datos

- La base de datos es el componente que se encarga de la persistencia de los datos que gestiona y manipula todo el Sistema.
- Es la "memoria permanente" del sistema.
- Sin ella, todo lo que pasa en el sistema se pierde al apagar el servidor.
- Se considera que la base de datos está dentro de la capa de persistencia de datos, pero ésta a su vez está dentro del Backend.
- Tecnologías típicas: PostgreSQL, MySQL, MongoDB, Oracle

Diagramas de Componentes y Despliegue

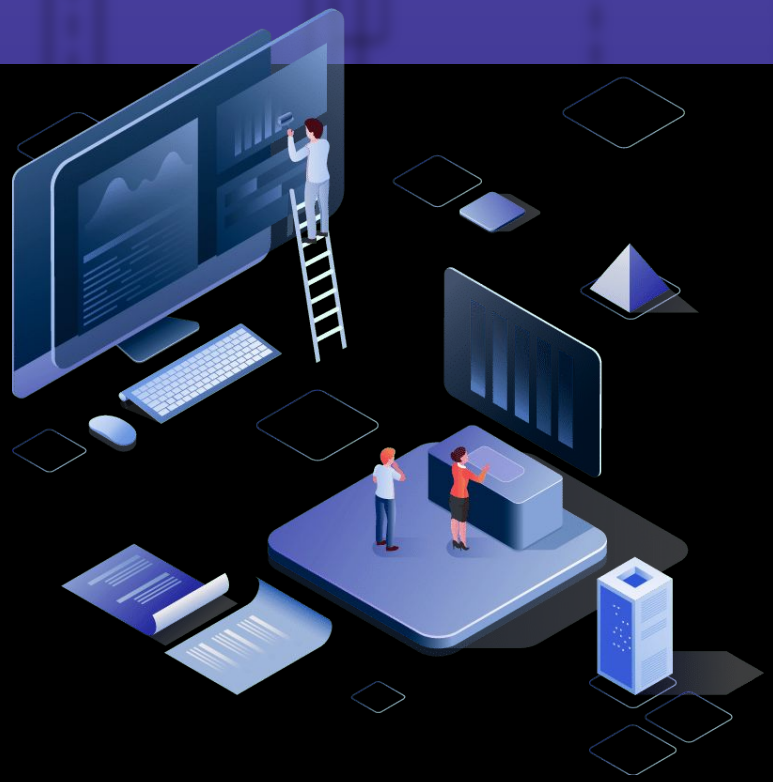


Diagrama de Componentes

¿Cómo se representa un Componente de Software en UML?

Un componente se representa como un rectángulo, con dos pequeños rectángulos superpuestos perpendicularmente en el lado izquierdo.

El Diagrama de Componentes tiene un nivel más alto de abstracción que un Diagrama de Clases.

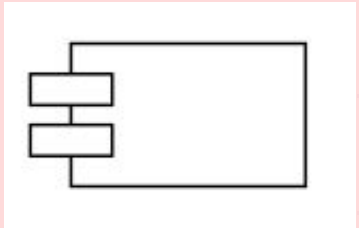


Diagrama de Componentes

Interfaz entre Componentes

El Componente 2 disponibiliza una interfaz que “consume” el Componente 1.

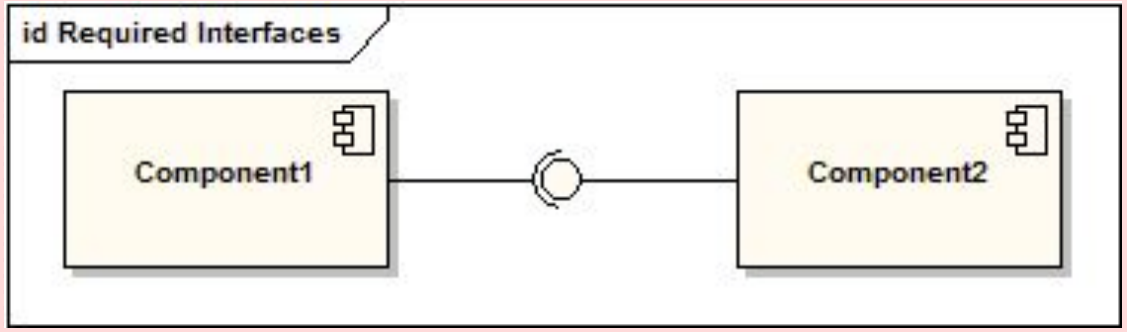


Diagrama de Componentes

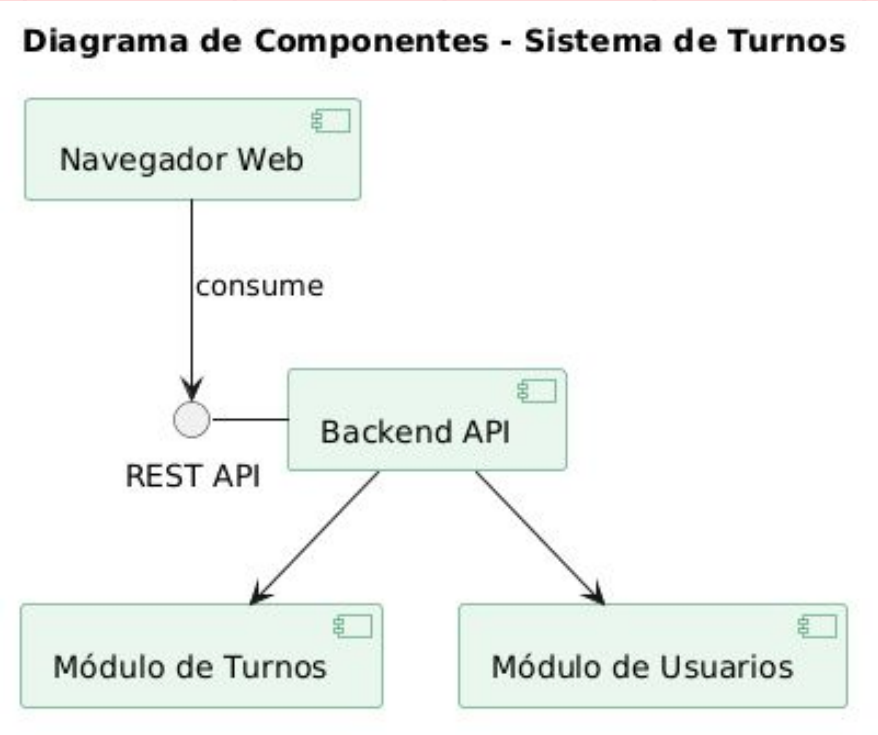


Diagrama de Componentes

¿Qué muestra este diagrama?

- Los componentes principales del Sistema y cómo se relacionan entre sí.
- La interfaz (REST API) que separa al Frontend del Backend — nadie habla "directamente", siempre hay un contrato
- Los módulos internos del Backend como subcomponentes
- No habla de servidores ni infraestructura — eso es tarea del diagrama de despliegue

Diagrama de Despliegue

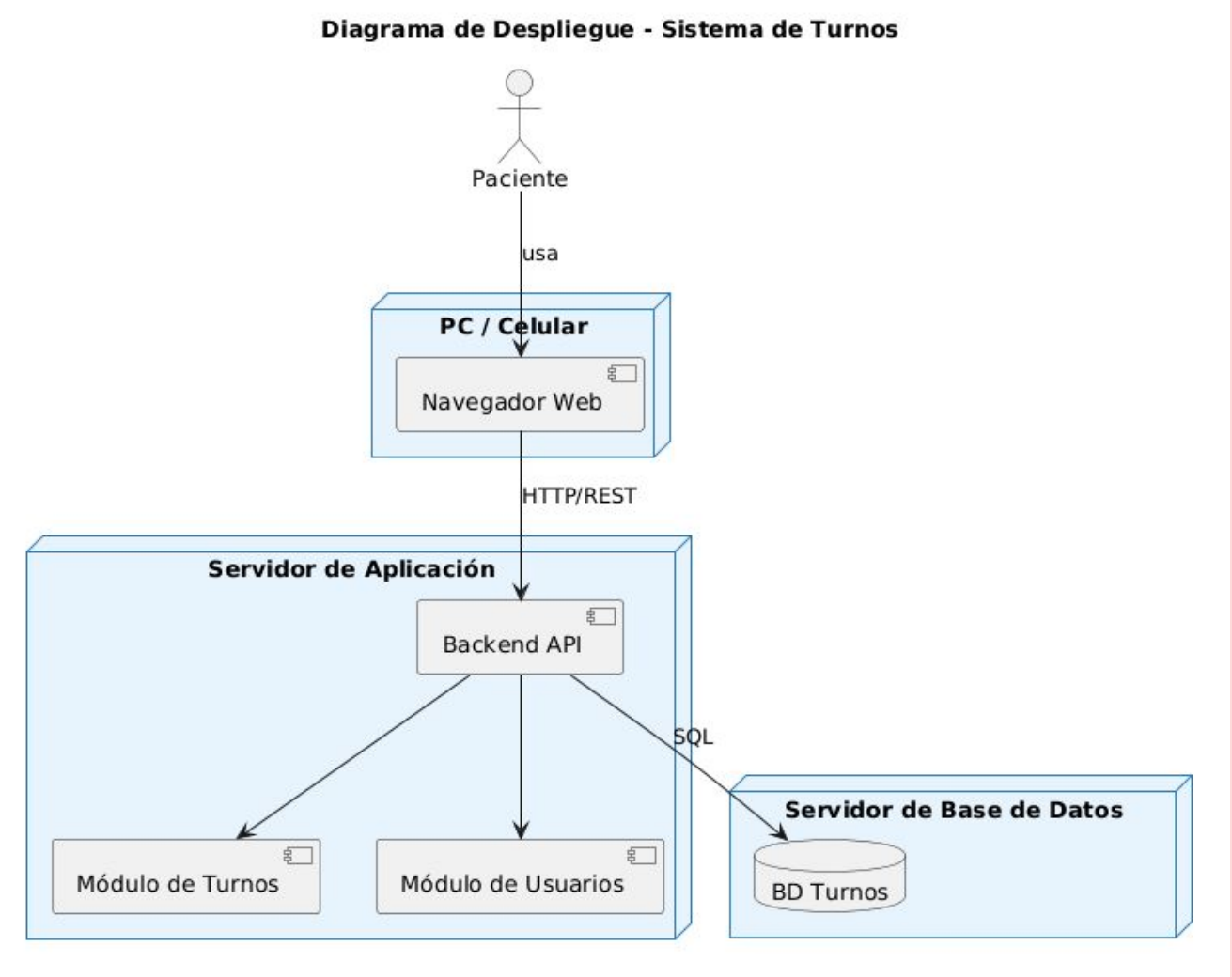


Diagrama de Despliegue

¿Qué agrega este diagrama respecto al anterior?

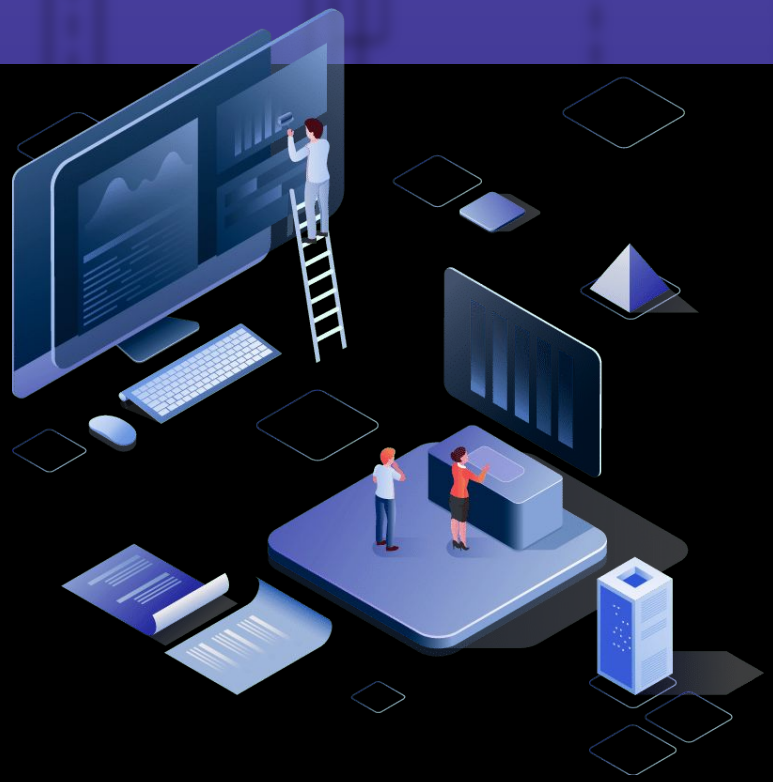
- Muestra dónde vive físicamente cada componente (en qué servidor o dispositivo)
- Permite ver cómo fluye la comunicación entre nodos (HTTP, etc.)
- Ayuda a identificar riesgos de infraestructura: ¿qué pasa si el servidor de BD se cae?
- Es útil para hablar con el cliente sobre costos de hosting, seguridad y escalabilidad

Diagrama de Despliegue vs Componentes

¿Qué agrega este diagrama respecto al anterior?

- Componentes → responde ¿qué piezas de software existen y cómo se conectan?
- Despliegue → responde ¿dónde corre cada pieza y a través de qué red se comunican?
- Se usan juntos: primero se entiende qué construir (componentes), después dónde desplegarlo (despliegue)
- En proyectos reales, el diagrama de despliegue lo revisa también el equipo de infraestructura / DevOps

Ejemplo Integrador



Ejemplo integrador

Escenario

Una clínica quiere digitalizar la gestión de turnos. Hoy todo se hace por teléfono y en papel. Nos contratan para diseñar el sistema.

Veamos qué preguntas podría hacerse el arquitecto del proyecto...

Ejemplo integrador

1) ¿Cuáles son las entradas?

- **RF**: Pacientes sacan turnos online, médicos ven su agenda, administración genera reportes
- **RNF**: Debe funcionar en el celular, soportar 500 turnos diarios, estar disponible 24/7
- **Restricción técnica**: El cliente ya tiene un servidor propio (no quiere nube)
- **Restricción de negocio**: Presupuesto acotado, debe estar listo en 3 meses

Ejemplo integrador

2) ¿Cuáles son los componentes típicos?

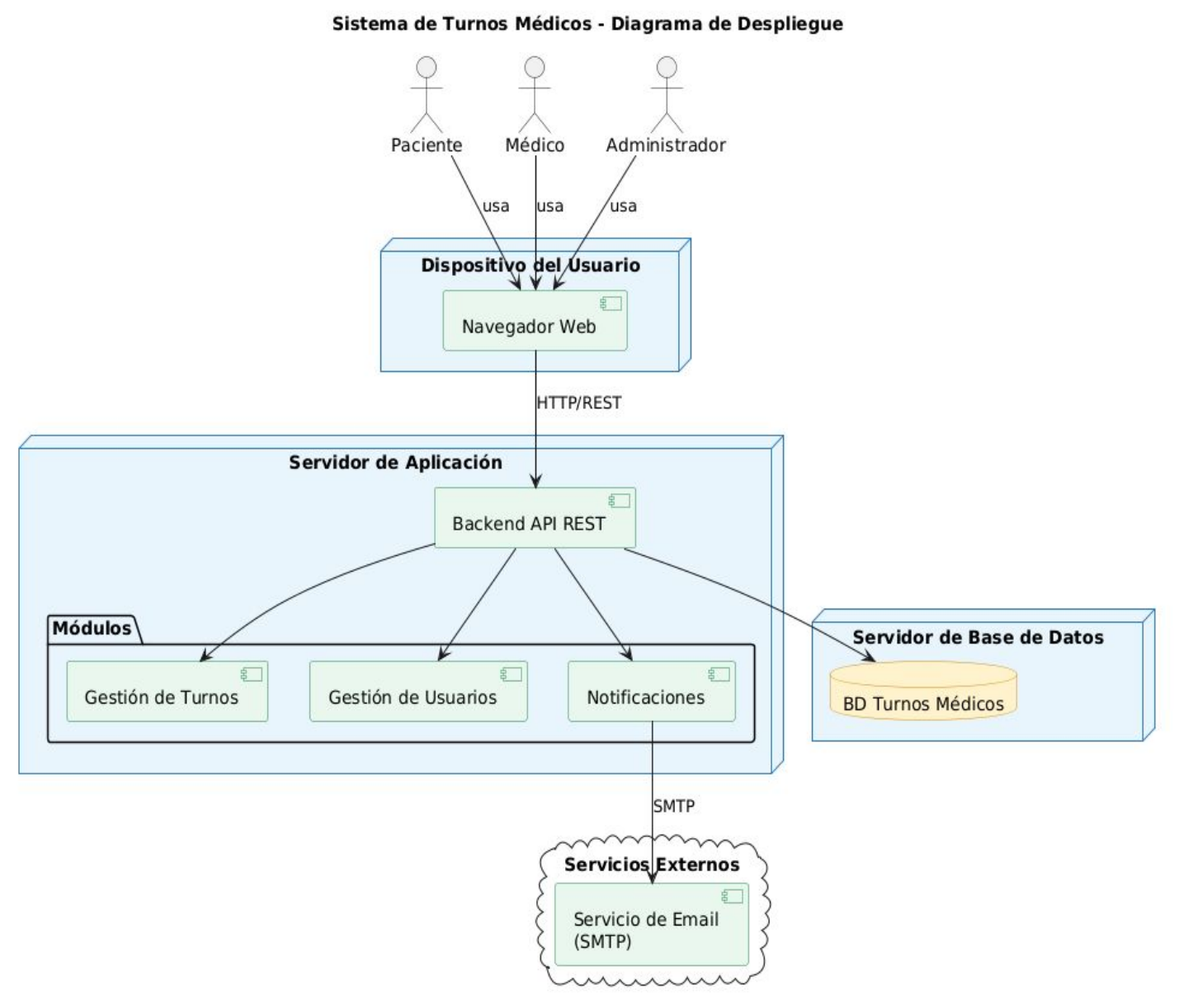
- **Frontend web:** para que los pacientes saquen turnos desde el navegador
- **Frontend mobile** (futuro): app para médicos
- **Backend**: gestiona la lógica de turnos, notificaciones, disponibilidad
- **Base de datos**: almacena pacientes, médicos, turnos, historiales

Ejemplo integrador

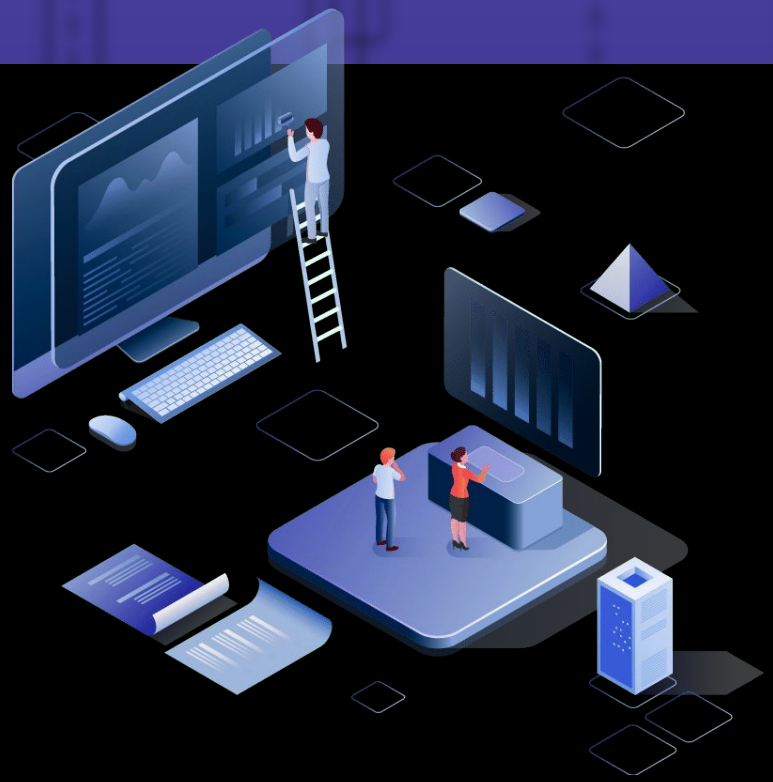
3) ¿Qué decisiones arquitectónicas hay que tomar?

- ¿Monolito o microservicios? (dado el presupuesto y el tiempo: monolito)
- ¿Cómo se notifica al paciente? (email, SMS, WhatsApp)
- ¿Cómo acceden los médicos? (web o app?)

Propuesta


Sistema de Turnos Médicos - Diagrama de Despliegue
Paciente
Médico
Administrador
usa
usa
usa
Dispositivo del Usuario
Navegador Web
HTTP/REST
Servidor de Aplicación
Backend API REST
Módulos
Gestión de Turnos
Gestión de Usuarios
Notificaciones
Servidor de Base de Datos
BD Turnos Médicos
SMTP
Servicios Externos
Servicio de Email
(SMTP)

Para Reflexionar



Para Reflexionar

¿Qué piensan de los trámites en el ámbito Universitario?

Para Reflexionar

¿Qué piensan de los trámites en el ámbito Universitario?

Ley de Conway

“Las organizaciones diseñan sistemas que replican su estructura de comunicación.”

Melvin Conway (1968). Científico Informático Estadounidense.

Ley de Conway

Caso: Gestión de Trámites - Excepción de Correlativas

- Proceso fragmentado entre áreas (Alumnos, Gestión Académica, Dpto. de Sistemas)
- Sistemas y formularios separados por sector
- Reingreso y duplicación de datos
- El alumno termina integrando el proceso
- Demoras por falta de visión end-to-end

El problema no es solo tecnológico → es organizacional
El sistema refleja cómo se comunican (o no) las áreas

Ley de Conway

Si tuvieran que mejorar los trámites en el ámbito universitario,
¿empezarían por el sistema o por la forma en que se organizan y comunican las áreas?

Ley de Conway - Conclusión

Si tuvieran que mejorar los trámites en el ámbito universitario,
¿empezarían por el sistema o por la forma en que se organizan y comunican las áreas?

La clave es diseñar procesos integrados end-to-end y, a partir de ahí, construir sistemas que acompañen esa lógica, poniendo en el centro la experiencia de la persona usuaria (estudiante/docente/no docente).

Gracias

